

CS 301

High-Performance Computing

Lab 02

Matrix Multiplication Performance Analysis

Submitted By

Dhanush Balachandran Pillai (202301483)
Jay Manishkumar Patel (202301430)

Course Instructor:
Bhaskar Chaudhury

Date of Submission:
February 17, 2026

Contents

1	Introduction	2
2	Hardware Details	3
2.1	Hardware Details for LAB207 PCs	3
2.2	Hardware Details for HPC Cluster (Node gics1)	3
3	Problem Description	4
4	Benchmarking Methodology	4
5	Graphical Results	5
5.1	Problem A-1: Conventional Matrix Multiplication	5
5.1.1	Throughput using Lab PC	5
5.1.2	Throughput using HPC Cluster	7
5.2	Problem B-1: Matrix Multiplication using Transpose	8
5.2.1	Throughput using Lab PC	8
5.2.2	Throughput using HPC Cluster	8
5.3	Problem C-1: Block Matrix Multiplication	9
5.3.1	Throughput using Lab PC	9
5.3.2	Throughput using HPC Cluster	9
6	Conclusion	10

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing and high-performance applications. The primary objective of this lab is to evaluate the performance of different matrix multiplication algorithms on two computing platforms: the Lab PC and the HPC Cluster.

This lab focuses on three specific methods:

- **Problem A-1:** Conventional matrix multiplication, evaluating all six possible loop orderings.
- **Problem B-1:** Matrix multiplication using the transpose method to improve data locality.
- **Problem C-1:** Block (or tiled) matrix multiplication using a divide-and-conquer approach to maximize cache utilization.

We measure and compare **runtime**, **throughput**, and **MFLOPS** across matrix sizes from 2^2 to 2^{12} , using both 4-byte integers and 8-byte floating-point values. This analysis will highlight the impact of memory hierarchy, algorithmic optimization, and hardware capabilities on performance.

2 Hardware Details

2.1 Hardware Details for LAB207 PCs

Table 1: Hardware Specifications of Lab 207 PCs

Specification	Details
Architecture	x86_64
CPU(s)	12
Threads per core	2
Cores per socket	6
Socket(s)	1
CPU Model	12th Gen Intel(R) Core(TM) i5-12500
CPU Max MHz	4600
CPU Min MHz	800
L1d Cache	288 KB
L1i Cache	192 KB
L2 Cache	7.5 MB
L3 Cache	18 MB

2.2 Hardware Details for HPC Cluster (Node gics1)

Table 2: Hardware Specifications of HPC Cluster Node gics1

Specification	Details
Architecture	x86_64
CPU(s)	16
Threads per core	1
Cores per socket	8
Socket(s)	2
CPU Model	Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
CPU MHz	1288.73
L1d Cache	32 KB
L1i Cache	32 KB
L2 Cache	256 KB
L3 Cache	20 MB

3 Problem Description

This lab implements three matrix multiplication approaches to investigate computational and memory performance:

1. **Conventional Matrix Multiplication (Problem A-1):** Computes the product using nested loops in all possible orderings. This highlights the effect of loop order on cache performance and runtime.
2. **Matrix Multiplication using Transpose (Problem B-1):** Improves spatial locality by transposing one of the matrices, thereby reducing cache misses and improving throughput.
3. **Block Matrix Multiplication (Problem C-1):** Divides matrices into smaller blocks and multiplies them using a divide-and-conquer strategy. This method exploits temporal locality to maximize cache utilization.

The goals of this lab are to:

- Analyze the effect of matrix size on **runtime** and **MFLOPS**.
- Compare memory-bound versus compute-bound behavior of each algorithm.
- Examine the performance improvement achieved by algorithmic optimizations.

4 Benchmarking Methodology

All algorithms were executed on both the Lab PC and the HPC Cluster. The methodology includes:

- Measuring **execution time** for various matrix sizes.
- Computing **throughput** using:

$$\text{Throughput} = \frac{\text{sizeof(element)} \times N^2 \times \text{operations}}{\text{Runtime (s)}}$$

- Calculating **MFLOPS** using:

$$\text{MFLOPS} = \frac{2N^3}{\text{Runtime (s)} \times 10^6}$$

All experiments were repeated five times to capture runtime fluctuations and support statistical analysis. The results are analyzed in terms of memory-bound vs compute-bound characteristics, and the effects of cache optimizations are highlighted.

5 Graphical Results

5.1 Problem A-1: Conventional Matrix Multiplication

5.1.1 Throughput using Lab PC

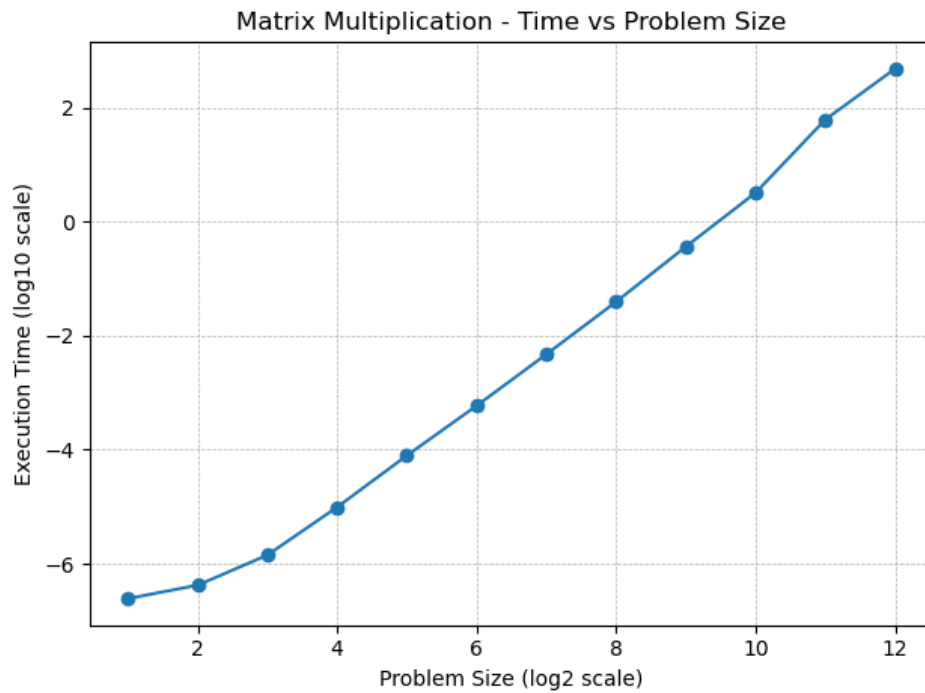


Figure 1: Throughput vs Problem Size for Conventional Multiplication **looping IJK** on Lab PC

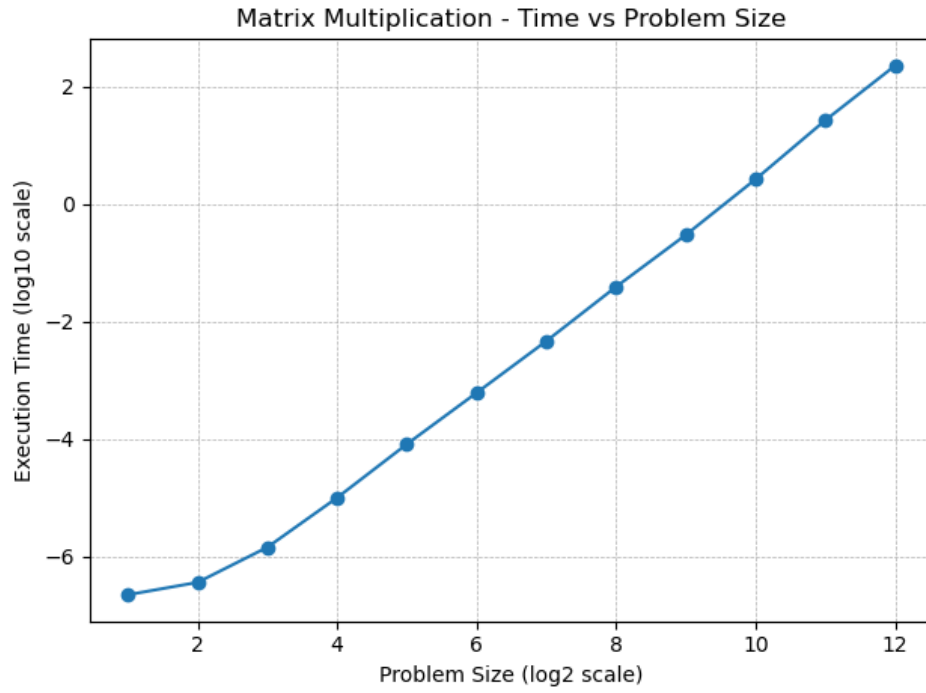


Figure 2: Throughput vs Problem Size for Conventional Multiplication **looping JIK** on Lab PC

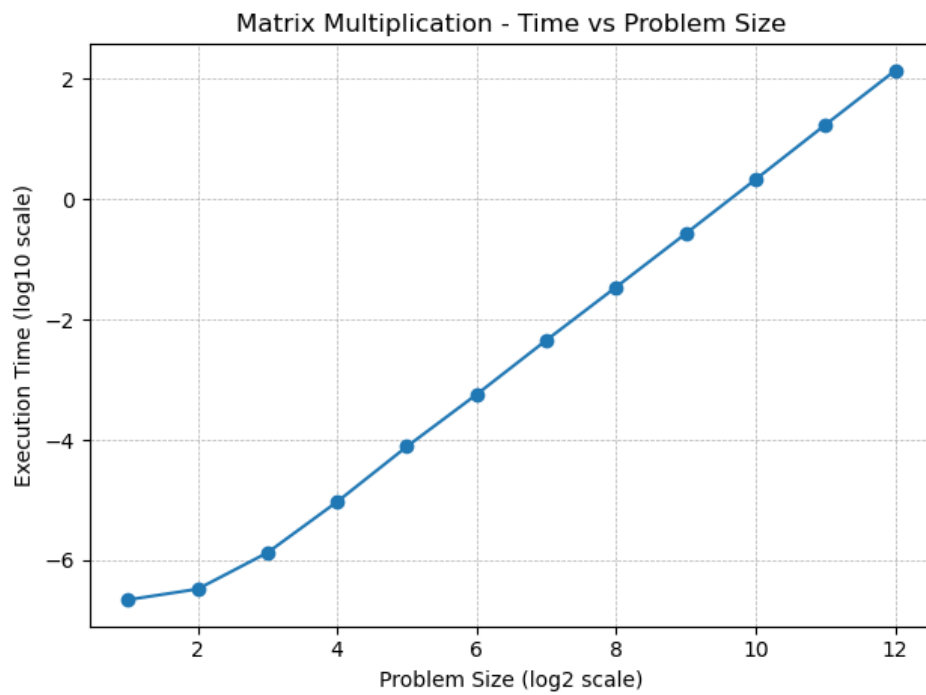


Figure 3: Throughput vs Problem Size for Conventional Multiplication **looping IKJ** on Lab PC

5.1.2 Throughput using HPC Cluster

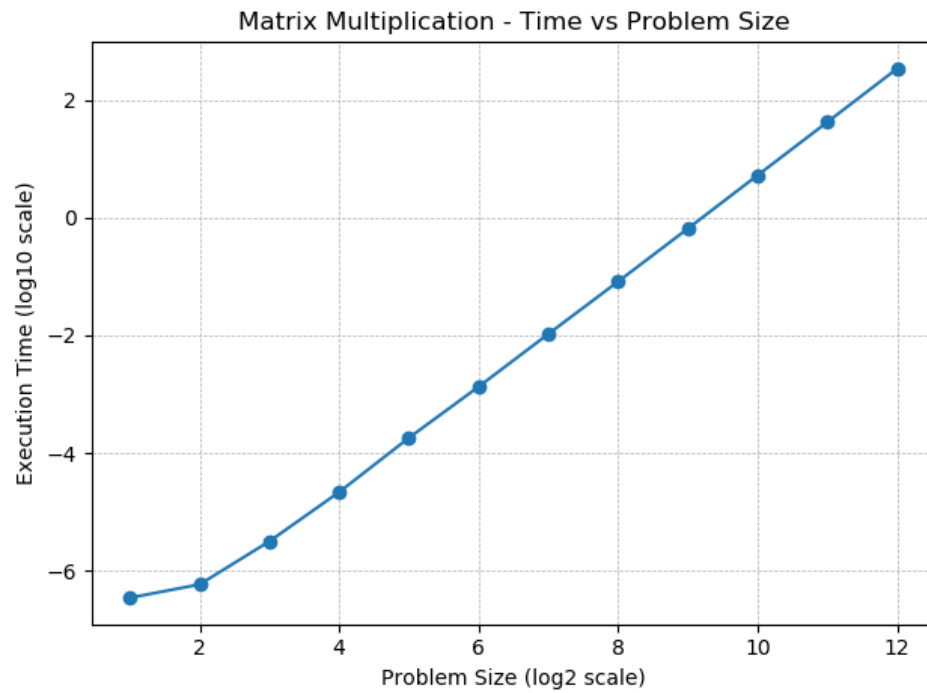


Figure 4: Throughput vs Problem Size for Conventional Multiplication **looping IKJ** on HPC Cluster

5.2 Problem B-1: Matrix Multiplication using Transpose

5.2.1 Throughput using Lab PC

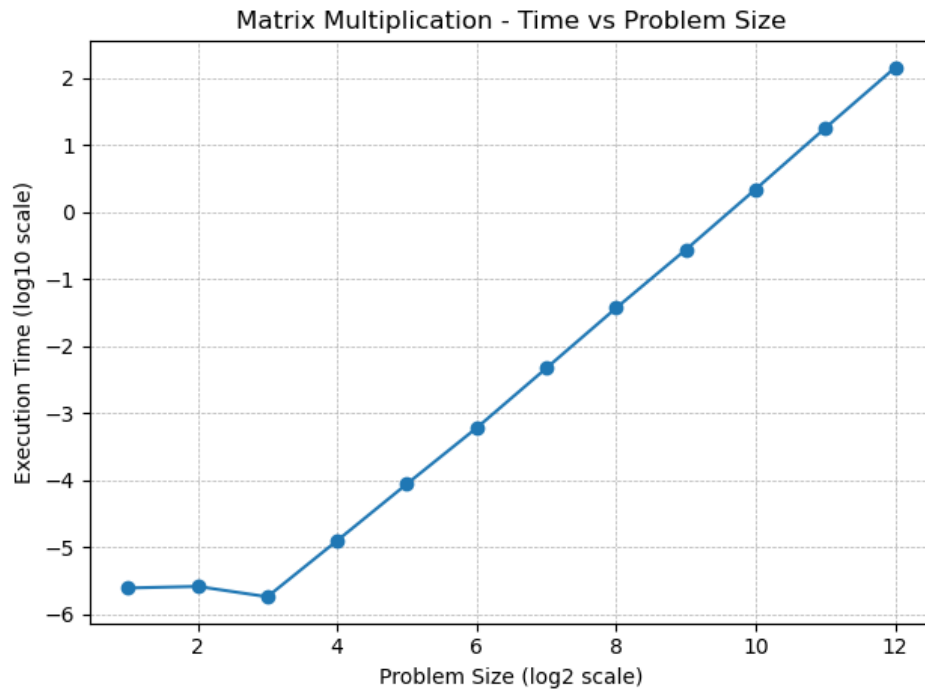


Figure 5: Throughput vs Problem Size for Transpose Multiplication on Lab PC

5.2.2 Throughput using HPC Cluster

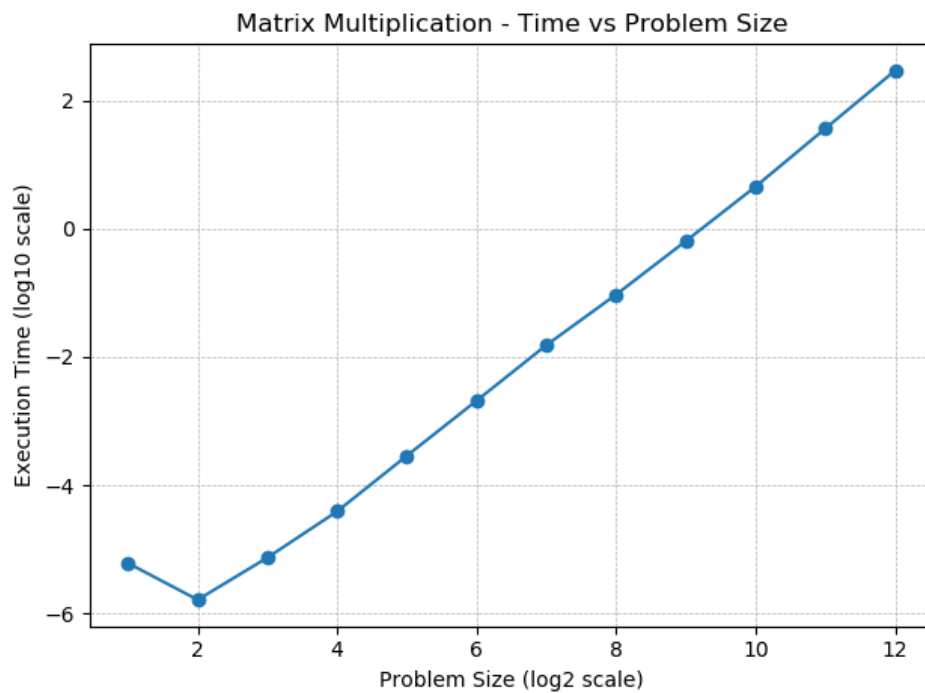


Figure 6: Throughput vs Problem Size for Transpose Multiplication on HPC Cluster

5.3 Problem C-1: Block Matrix Multiplication

5.3.1 Throughput using Lab PC

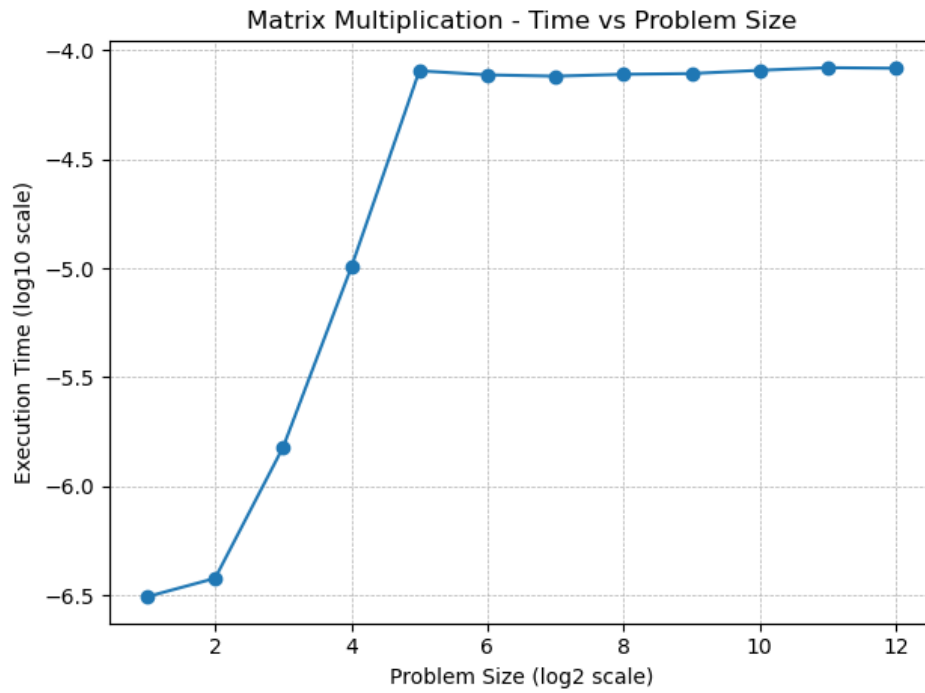


Figure 7: Throughput vs Problem Size for Block Multiplication on Lab PC

5.3.2 Throughput using HPC Cluster

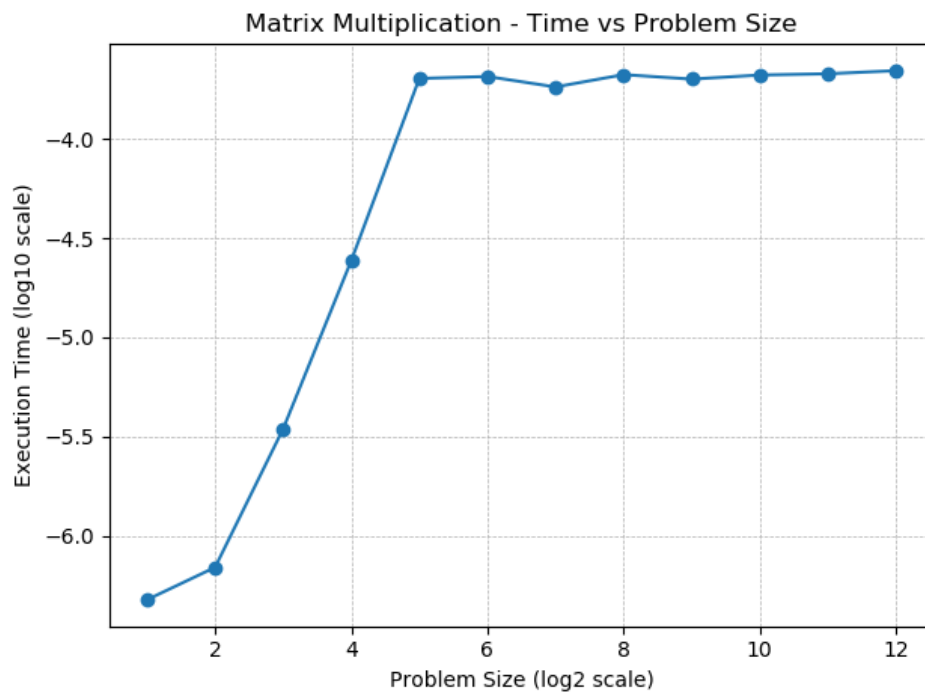


Figure 8: Throughput vs Problem Size for Block Multiplication on HPC Cluster

6 Conclusion

- Runtime increases with matrix size ($O(N^3)$ complexity).
- Conventional multiplication shows lower throughput for large matrices due to cache misses.
- Transpose multiplication improves throughput by enhancing spatial locality.
- Block multiplication achieves highest throughput due to efficient cache reuse.
- HPC Cluster outperforms Lab PC due to more cores and larger caches.
- Observed throughput is below the theoretical maximum due to memory bandwidth limitations.